

Yes, `layout.tsx` and `template.tsx` can and should be used together in Next.js, especially with Framer Motion, to achieve persistent layouts with dynamic page transition animations.  LinkedIn +1

How They Work Together

- **`layout.tsx` for Persistence:** The `layout.tsx` file defines UI that persists across routes and does not remount, such as a header, sidebar, or footer. This component is rendered once and maintains its state, making it ideal for the outer shell of your application.
- **`template.tsx` for Transitions:** The `template.tsx` file is similar to a layout but creates a **new instance** of its children on every navigation. This behavior is crucial for Framer Motion, as it forces the component to remount, triggering entry (and potentially exit) animations for the new page content.  DEV Community +4

Best Practices for Framer Motion

To implement effective page transitions in the Next.js App Router using Framer Motion, follow these steps:

1. **Use `template.tsx` for Animation Wrappers:** Place your Framer Motion animation logic (e.g., `motion.div` with `initial`, `animate`, and `transition` props) inside a `template.tsx` file in the relevant directory (e.g., the root `app/` directory for global transitions).
2. **Add "use client" :** Since Framer Motion involves client-side interactions and hooks, the component containing the animation logic must be a Client Component. Add `"use client";` at the top of your `template.tsx` file.
3. **Combine with `layout.tsx` :** The `layout.tsx` (which is a Server Component by default) will wrap the `template.tsx`. This structure allows the outer layout (like a navbar) to remain static and persistent while the inner content (the page wrapped by the template) animates on navigation.
4. **Consider `AnimatePresence` Workarounds (for exit animations):** While `template.tsx` handles entry animations well, implementing exit animations with `AnimatePresence` can be complex due to how the App Router handles shared layouts. The community has developed specific workarounds, often involving wrapping the content with a "frozen" router context, to enable full exit animations.  Stack Overflow +4

By leveraging both `layout.tsx` and `template.tsx`, you can create a structured and performant Next.js application with smooth, professional page transitions. The Next.js documentation provides further details on [Layouts vs Templates](#) file conventions.